

C2SafeRust-LLM

Module 6: Compiler Validation & LLM Repair Loop -- Line-by-Line Explanation

For Beginners | Mini Project S3

Module 6: Compiler Validation & LLM Repair Loop

Line-by-Line Code Explanation for Beginners

Files: mod.rs, compiler_loop.rs, diagnostic_packager.rs, llm_client.rs, patch_applier.rs

What does Module 6 do? Module 5 produced a CANDIDATE Rust file — code that SHOULD be valid Rust, but may have mistakes because automated translation is hard. Module 6 runs a FEEDBACK LOOP: 1. Compile the candidate with `rustc` 2. If it compiles: SUCCESS! Return the working code. 3. If it fails: Send the errors + original C + broken Rust to the Gemini LLM API 4. The LLM returns a REPAIRED version of the Rust code 5. Apply the repair to the file and go back to step 1 6. Repeat up to `max_iterations` times This "compile → diagnose → repair → retry" cycle is the KEY INNOVATION of C2SafeRust-LLM. It uses AI to do what no static rule system can do alone.

The Module 6 Repair Loop:



File 1: src/module6_repair/mod.rs

(The Repair Loop Orchestrator)

Lines 1-9: Module declarations and re-exports

```
pub mod compiler_loop;
pub mod diagnostic_packager;
pub mod llm_client;
pub mod patch_applier;

pub use compiler_loop::{invoke_rustc_compiler, CompilationResult};
pub use diagnostic_packager::{package_diagnostics, RepairRequestPackage};
pub use llm_client::{query_llm_repair_agent, PatchProposal};
pub use patch_applier::{apply_patch_to_file, RepairLogEntry};
```

Four sub-modules, each handling exactly one step of the loop:

- `compiler_loop` → runs `rustc`, parses JSON diagnostics
- `diagnostic_packager` → bundles inputs into a payload for the LLM
- `llm_client` → calls the Gemini API (or fallback)
- `patch_applier` → writes the repaired code back to disk and logs it

Lines 11-17: The Output Struct `RepairResult`

```
#[derive(Debug, Clone)]
pub struct RepairResult {
    pub final_success: bool,
    pub iterations_used: usize,
    pub repair_logs: Vec<RepairLogEntry>,
    pub final_rust_code: String,
}
```

Field	Type	What it holds
<code>final_success</code>	<code>bool</code>	<code>true</code> if the code compiled successfully at any iteration.
<code>iterations_used</code>	<code>usize</code>	How many repair iterations were needed (1 = compiled on first try).
<code>repair_logs</code>	<code>Vec</code>	One log entry per repair attempt — stored for the GUI and report.
<code>final_rust_code</code>	<code>String</code>	The Rust code as it stands after the last successful repair (or the best attempt).

Lines 19-71: The Main Repair Loop `run_repair_loop()`

```
pub fn run_repair_loop(
    c_source: &str,
    candidate_rust_path: &str,
    max_iterations: usize,
) -> Result<RepairResult, String> {
```

Parameter	Type	What it receives
`c_source`	`&str`	The original C source code text — sent to the LLM as context.
`candidate_rust_path`	`&str`	Path to the `.rs` file Module 5 generated (e.g., `output/output.rs`).
`max_iterations`	`usize`	How many repair cycles to attempt before giving up (typically 3–5).

Lines 25-28: Read the Initial Candidate

```
let mut logs = Vec::new();
let mut current_rust = std::fs::read_to_string(candidate_rust_path)
    .map_err(|e| format!(
        "Failed to read candidate Rust file '{}': {}",
        candidate_rust_path, e
    ))?;
```

	Explanation		
	Empty log list — will grow by one entry per repair iteration.		
	Read the initial generated `.rs` file into memory. `mut` because it will be updated each iteration.		
	Full path used instead of `use std::fs` at the top — both work, but full paths avoid import clutter.		
	e\	format!(...))`	Convert `io::
	If the file can't be read (doesn't exist, permissions), return `Err` immediately.		

Lines 29-59: The Core Loop

```

for iteration in 1..=max_iterations {
    println!("    Compiler Loop Iteration {}/{}...", iteration, max_iterations);

    let comp_res = invoke_rustc_compiler(candidate_rust_path);

    if comp_res.success {
        println!("    Compilation Succeeded at Iteration {}!", iteration);
        current_rust = std::fs::read_to_string(candidate_rust_path)
            .unwrap_or(current_rust);
        return Ok(RepairResult {
            final_success: true,
            iterations_used: iteration,
            repair_logs: logs,
            final_rust_code: current_rust,
        });
    }

    println!("    rustc rejected candidate code with {} errors.", comp_res.error_count);

    let pkg      = package_diagnostics(c_source, &current_rust, &comp_res.diagnostics, iteration);
    let proposal = query_llm_repair_agent(&pkg)?;
    let mut log_entry = apply_patch_to_file(candidate_rust_path, &proposal, iteration)?;
    log_entry.compilation_successful = false;
    logs.push(log_entry);

    current_rust = proposal.patched_rust_code;
}

```

Key Rust features in this loop:

Code	Explanation
<code>`for iteration in 1..=max_iterations`</code>	<code>`1..=max_iterations`</code> is an INCLUSIVE range — includes both <code>`1`</code> AND <code>`max_iterations`</code> . Contrast with <code>`1..max_iterations`</code> which excludes the last value.
<code>`if comp_res.success { return Ok(...); }`</code>	Early return inside the loop — exits the function immediately if compilation succeeds. No <code>`break`</code> needed.
<code>`.unwrap_or(current_rust)`</code>	Re-read the file after success. If re-reading fails, use the in-memory <code>`current_rust`</code> as fallback. Note: <code>`current_rust`</code> is MOVED here — this is safe because the <code>`return`</code> follows immediately.
<code>`query_llm_repair_agent(&pkg)?`</code>	The <code>`?`</code> operator — if the LLM call fails (network error, API error), the ENTIRE <code>`run_repair_loop`</code> function returns <code>`Err(String)`</code> upward to the caller.
<code>`log_entry.compilation_successful = false`</code>	The patch applier sets this to <code>`false`</code> by default. We confirm it explicitly after knowing compilation failed.
<code>`current_rust = proposal.patched_rust_code`</code>	Update the in-memory rust code to the patched version for the next iteration's diagnostics.

Lines 61-70: Post-Loop Final Check

```
let final_comp = invoke_rustc_compiler(candidate_rust_path);
current_rust = std::fs::read_to_string(candidate_rust_path).unwrap_or(current_rust);

Ok(RepairResult {
    final_success: final_comp.success,
    iterations_used: max_iterations,
    repair_logs: logs,
    final_rust_code: current_rust,
})
```

If we used ALL iterations without a successful compile, we run one FINAL check. The result might still be `final_success: false` — meaning the LLM couldn't fix it in the allotted iterations. Module 7 reports this to the user with the repair log for manual review.

File 2: `src/module6_repair/compiler_loop.rs`

(Running rustc and Parsing JSON Diagnostics)

Why JSON diagnostics? By default, `rustc` prints error messages designed for humans to READ. But we need a MACHINE to read and process errors. The `--error-format=json` flag makes `rustc` output each error as a JSON object — structured data we can parse with `serde_json` into Rust structs.

Lines 1-2: Imports

```
use serde::{Deserialize, Serialize};
use std::process::Command;
```

Import	Why
<code>`serde::{Deserialize, Serialize}`</code>	The <code>`serde`</code> crate provides the serialization/deserialization framework. <code>`Deserialize`</code> lets us parse JSON → Rust struct automatically. <code>`Serialize`</code> lets us convert Rust struct → JSON.
<code>`std::process::Command`</code>	Run <code>`rustc`</code> as a child process — same pattern used in Module 2 for GCC.

Lines 4-24: The rustc Diagnostic Structs

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct RustcDiagnosticSpan {
    pub file_name: String,
    pub line_start: usize,
    pub line_end: usize,
    pub column_start: usize,
    pub column_end: usize,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct RustcDiagnosticCode {
    pub code: String, // e.g., "E0308" (mismatched types)
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct RustcDiagnosticMessage {
    pub message: String,
    pub code: Option<RustcDiagnosticCode>,
    pub level: String, // "error", "warning", "note"
    pub spans: Vec<RustcDiagnosticSpan>,
}
```

What is `#[derive(Serialize, Deserialize)]`? These are macros from the `serde` crate (listed in `Cargo.toml`). `Deserialize` generates code that can parse JSON like: ``json {"message": "mismatched types", "level": "error", "code": {"code": "E0308"}, "spans": [...]}`` ...directly into a `RustcDiagnosticMessage` struct, with no manual parsing code needed.

The actual JSON format `rustc --error-format=json` produces:

```
{
  "message": "mismatched types",
  "code": { "code": "E0308" },
  "level": "error",
  "spans": [{
    "file_name": "output.rs",
    "line_start": 5,
    "line_end": 5,
    "column_start": 10,
    "column_end": 15
  }]
}
```

Struct Field	JSON Key	Example Value
<code>`message`</code>	<code>`"message"`</code>	<code>`"mismatched types"`</code>
<code>`code`</code>	<code>`"code"`</code>	<code>`{ "code": "E0308" }`</code> — wrapped in another struct
<code>`level`</code>	<code>`"level"`</code>	<code>`"error"`, `"warning"`, or `"note"`</code>
<code>`spans`</code>	<code>`"spans"`</code>	Array of source locations where the error occurred

Why `Option<RustcDiagnosticCode>` for code?

Not all `rustc` messages have an error code. Warnings and notes don't always have one.

`Option<...>` handles this: `None` when no code is present, `Some(RustcDiagnosticCode{...})` when there is one.

Lines 26-32: The `CompilationResult` Struct

```
#[derive(Debug, Clone)]
pub struct CompilationResult {
    pub success: bool,
    pub raw_output: String,
    pub diagnostics: Vec<RustcDiagnosticMessage>,
    pub error_count: usize,
}
```

Field	What it holds
<code>`success`</code>	<code>`true`</code> if <code>`rustc`</code> exited with code 0 AND no errors were found in the JSON output.
<code>`raw_output`</code>	The complete raw stderr text from <code>`rustc`</code> — stored for debugging.
<code>`diagnostics`</code>	All parsed <code>`RustcDiagnosticMessage`</code> structs — passed to the diagnostic packager.
<code>`error_count`</code>	How many <code>`"level": "error"`</code> messages were found — shown in the GUI log.

Lines 36-72: The `invoke_rustc_compiler()` Function

```
pub fn invoke_rustc_compiler(rs_file_path: &str) -> CompilationResult {
    let output = Command::new("rustc")
        .arg("--error-format=json")
        .arg("--crate-type=lib")
        .arg(rs_file_path)
        .output();
}
```

Argument	Effect
<code>`"rustc"`</code>	Run the Rust compiler. Must be installed on the system PATH.
<code>`"--error-format=json"`</code>	Output diagnostics as one JSON object per line on stderr.
<code>`"--crate-type=lib"`</code>	Compile as a LIBRARY, not an executable. This means no <code>`fn main()`</code> is required, making it easier to test individual generated function files.
<code>`rs_file_path`</code>	The path to the <code>`.rs`</code> file to compile.
<code>`.output()`</code>	Run synchronously, capture all stdout and stderr. Returns <code>`Result`</code> .

Lines 43-71: Parsing the Compilation Output

```
match output {
  Ok(out) => {
    let stderr_str = String::from_utf8_lossy(&out.stderr).to_string();
    let mut diagnostics = Vec::new();
    let mut error_count = 0;

    for line in stderr_str.lines() {
      if let Ok(diag) = serde_json::from_str::(<RustcDiagnosticMessage>(line)) {
        if diag.level == "error" {
          error_count += 1;
        }
        diagnostics.push(diag);
      }
    }

    CompilationResult {
      success: out.status.success() && error_count == 0,
      raw_output: stderr_str,
      diagnostics,
      error_count,
    }
  }
  Err(err) => CompilationResult {
    success: false,
    raw_output: format!("Failed to execute rustc: {}", err),
    diagnostics: Vec::new(),
    error_count: 1,
  },
}
```

Step	Code	Explanation
<code>`match output`</code>	Pattern match on <code>`Result`</code>	<code>`Ok(out)`</code> = <code>`rustc`</code> launched successfully. <code>`Err(err)`</code> = <code>`rustc`</code> couldn't be started (not installed).
<code>`String::from_utf8_lossy(&out.stderr);`</code>	Read stderr	<code>`rustc`</code> JSON messages go to <code>`stderr`</code> . <code>`to_string()`</code> materializes the cow string.
<code>`for line in stderr_str.lines()`</code>	One line per diagnostic	With <code>`--error-format=json`</code> , each error/warning is on its own JSON line.
<code>`serde_json::from_str::(line)`</code>	Parse JSON → struct	Turbofish <code>`::`</code> tells <code>`serde_json`</code> WHICH struct to deserialize into. <code>`if let Ok(diag)`</code> — skip lines that aren't valid JSON (e.g., empty lines).
<code>`if diag.level == "error"`</code>	Count only errors	<code>`error_count`</code> only increments for actual errors, not warnings or notes.
<code>`out.status.success() && error_count == 0`</code>	Double success check	Even if <code>`rustc`</code> exits with code 0 but JSON contains errors, we mark it failed. Both conditions must be true.

Step	Code	Explanation
<code>`Err(err) => CompilationResult { error_count: 1 }`</code>	Cannot run rustc	If <code>`rustc`</code> isn't installed or on PATH, return a synthetic failure result.

The Unit Test (Lines 79-91)

```
#[test]
fn test_rustc_invocation_valid_code() {
    let test_file = "temp_valid.rs";
    fs::write(test_file, "pub fn add(a: i32, b: i32) -> i32 { a + b }").unwrap();

    let res = invoke_rustc_compiler(test_file);
    let _ = fs::remove_file(test_file);
    let _ = fs::remove_file("libtemp_valid.rlib");

    assert!(res.success);
    assert_eq!(res.error_count, 0);
}
```

Line	Explanation
<code>`fs::write(test_file, "pub fn add..."`</code>	Create a known-good Rust file on disk.
<code>`let _ = fs::remove_file("libtemp_valid.rlib")`</code>	When <code>`rustc --crate-type=lib`</code> compiles successfully, it produces a <code>`.rlib`</code> file. This cleans it up. The name is derived from the source file name: <code>`temp_valid.rs`</code> → <code>libtemp_valid.rlib`</code> .

File 3: src/module6_repair/diagnostic_packager.rs

(Building the LLM Repair Payload)

Why package diagnostics? The LLM needs CONTEXT to produce a good repair: - The ORIGINAL C code (what we were trying to translate) - The CANDIDATE Rust code (what we generated — the broken code) - The COMPILER ERRORS (what exactly is wrong) - The ITERATION NUMBER (how many attempts this is) This file bundles all of that into one struct ready to be serialized and sent.

Lines 1-11: Imports and the Package Struct

```

use serde::{Deserialize, Serialize};
use crate::module6_repair::compiler_loop::RustcDiagnosticMessage;

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct RepairRequestPackage {
    pub original_c_source: String,
    pub candidate_rust_code: String,
    pub compiler_error_messages: Vec<String>,
    pub structured_diagnostics: Vec<RustcDiagnosticMessage>,
    pub iteration_number: usize,
}

```

Field	What it holds
`original_c_source`	The full C source text — gives the LLM the "intended semantics".
`candidate_rust_code`	The broken Rust code — what the LLM needs to FIX.
`compiler_error_messages`	Human-readable error strings like "[E0308] mismatched types".
`structured_diagnostics`	The full parsed `RustcDiagnosticMessage` structs (for span information).
`iteration_number`	Which repair attempt this is. The LLM can use this to know if previous repairs failed.

Why **Serialize** AND **Deserialize** on this struct?

Serialize so we could serialize it to JSON for logging or caching.

Deserialize so it could theoretically be read back from a log file.

In practice, we build it in memory and pass it directly to the LLM client.

Lines 14-36: The package_diagnostics() Function

```
pub fn package_diagnostics(
    c_source: &str,
    rust_code: &str,
    diagnostics: &[RustcDiagnosticMessage],
    iteration: usize,
) -> RepairRequestPackage {
    let error_messages: Vec<String> = diagnostics
        .iter()
        .map(|d| {
            let code_str = d.code
                .as_ref()
                .map(|c| c.code.clone())
                .unwrap_or_default();
            format!("[{}] {}", code_str, d.message)
        })
        .collect();

    RepairRequestPackage {
        original_c_source: c_source.to_string(),
        candidate_rust_code: rust_code.to_string(),
        compiler_error_messages: error_messages,
        structured_diagnostics: diagnostics.to_vec(),
        iteration_number: iteration,
    }
}
```

Parameter	Type	Note
<code>`diagnostics: &[RustcDiagnosticMessage]`</code>	Slice reference	<code>`&[T]`</code> is a slice — a reference to a contiguous sequence. More general than <code>`&Vec`</code> since it works with both <code>`Vec`</code> and arrays.

Building human-readable error strings:

```
diagnostics.iter().map(|d| {
    let code_str = d.code
        .as_ref() // Option<RustcDiagnosticCode> -> Option<&RustcDiagnosticCode>
        .map(|c| c.code.clone()) // Option<&Code> -> Option<String> (extract "E0308")
        .unwrap_or_default(); // Option<String> -> String (" " if None)
    format!("[{}] {}", code_str, d.message)
})
```

Input	Output	Explanation		
<code>`Option`</code>	—	The diagnostic's optional error code.		
<code>`Option`</code>	<code>`Option<&RustcDiagnosticCode>`</code>	Borrow the inside without moving. Needed to chain <code>`map()`</code> without consuming <code>`d`</code> .		

	Input	Output	Explanation		
	c\	c.code.clone())`	`Option<&Code;>`	`Option`	Extract the `c` from the struct
default())`	`Option`	`String`	If `None` (no error code), returns `` (empty String default).		
], {}", message)`	—	""[E0308] mismatched types""	Final readable line for the LLM prompt.		

`diagnostics.to_vec()` — converts a `&[T]` slice to an owned `Vec<T>` by cloning all elements.

File 4: `src/module6_repair/llm_client.rs`

(Calling the Gemini API — The AI Repair Engine)

This is the most "exciting" file — it's where the code actually talks to the Gemini LLM. It follows a two-path approach: Path A: Call the real Gemini API if an API key is available Path B: Fall back to rule-based heuristics if the API is unavailable

Lines 1-10: Imports and PatchProposal

```
use std::env;
use serde_json::json;
use crate::module6_repair::diagnostic_packager::RepairRequestPackage;

#[derive(Debug, Clone)]
pub struct PatchProposal {
    pub patched_rust_code: String,
    pub explanation: String,
    pub source: String, // "Gemini-LLM-API" or "Rule-Based-Fallback"
}
```

Import	Why
<code>std::env</code>	Read environment variables — specifically <code>GEMINI_API_KEY</code> .
<code>serde_json::json</code>	A macro that creates a <code>serde_json::Value</code> from JSON-like syntax directly in Rust code.

`PatchProposal.source` distinguishes whether the fix came from the real AI or the fallback — shown in the GUI repair log.

Lines 12-34: The Dispatcher `query_llm_repair_agent()`

```
pub fn query_llm_repair_agent(
    pkg: &RepairRequestPackage
) -> Result<PatchProposal, String> {
    // Try to get API key from environment variable first
    let api_key = env::var("GEMINI_API_KEY")
        .ok()
        .or_else(|| read_key_from_dotenv());

    if let Some(key) = api_key {
        if !key.trim().is_empty() {
            println!("    Sending diagnostic package to Gemini LLM Repair API...");
            match call_gemini_api(&key, pkg) {
                Ok(proposal) => return Ok(proposal),
                Err(err) => {
                    println!("    Gemini API issue: {}. Using fallback.", err);
                }
            }
        }
    }

    println!("    Using rule-based repair agent fallback...");
    Ok(fallback_rule_based_repair(pkg))
}
```

	Explanation		
	Read the environment variable. Returns `Result`.		
	Convert `Result` → `Option`. `Ok(s)` → `Some(s)`, `Err(_)` → `None`.		
	\	read_key_from_dotenv())`	If `.ok()` gave `.env` file. `o the option is
	Only proceed if we actually got a key.		
	Guard against an empty string key (e.g., `GEMINI_API_KEY=` with no value).		
), Err(err) =>	Try the API. If it succeeds, return immediately. If it fails (network down, quota exceeded), fall through to the fallback.		
	Fallback always succeeds (no network needed), so wrap in `Ok`.		

Lines 36-49: Reading API Key from .env File

```
fn read_key_from_dotenv() -> Option<String> {
    if let Ok(content) = std::fs::read_to_string(".env") {
        for line in content.lines() {
            let trimmed = line.trim();
            if trimmed.starts_with("GEMINI_API_KEY=") {
                let val = trimmed
                    .trim_start_matches("GEMINI_API_KEY=")
                    .trim();
                if !val.is_empty() {
                    return Some(val.to_string());
                }
            }
        }
    }
    None
}
```

A `.env` file looks like this: `` GEMINI\_API\_KEY=AizaSy...yourkey... SOME\_OTHER\_KEY=value ``

Code	Explanation
<code>`std::fs::read_to_string(".env")`</code>	Read the <code>.env</code> file in the current working directory. Returns <code>Result</code> — if file doesn't exist, <code>if let Ok(...)</code> simply skips the block.
<code>`trimmed.starts_with("GEMINI_API_KEY=")`</code>	Find the line with the API key.
<code>`.trim_start_matches("GEMINI_API_KEY=")`</code>	Remove the key name and <code>`=`</code> prefix, leaving just the API key value.
<code>`.trim()`</code>	Remove any trailing whitespace or newline chars.

Lines 51-107: The Real Gemini API Call

```
fn call_gemini_api(
    api_key: &str,
    pkg: &RepairRequestPackage
) -> Result<PatchProposal, String> {
    let url = format!(
        "https://generativelanguage.googleapis.com/v1beta/models/gemini-3.6-flash:generateContent?api_key={api_key}"
    );
}
```

Building the LLM Prompt:

```
let prompt = format!(
    "You are an expert Rust compiler repair agent for C-to-Rust transpilation.\n\
    The following Rust code was synthesized from C, but failed to compile with rustc.\n\n\
    === Original C Source ===\n{}\n\n\
    === Candidate Rust Code ===\n{}\n\n\
    === rustc Diagnostic Errors ===\n{}\n\n\
    Please provide the REPAIRED valid Rust code only enclosed in ``rust ... `` blocks, \
    preserving memory safety and ownership intent.",
    pkg.original_c_source,
    pkg.candidate_rust_code,
    pkg.compiler_error_messages.join("\n")
);
```

The prompt has **THREE** sections that give the LLM everything it needs: 1. Original C → "what was the programmer's intent?" 2. Candidate Rust → "what did the transpiler produce?" 3. rustc errors → "what exactly is broken?" The instruction "enclosed in ``rust...`` blocks" tells the LLM to format its response in a way our `extract_rust_code_block()` parser can find.

Building the JSON Request Body:

```
let payload = json!({
    "contents": [{
        "parts": [{ "text": prompt }]
    }]
});
```

The `json!()` macro creates this Rust value (a `serde_json::Value`):

```
{
  "contents": [
    {
      "parts": [
        { "text": "You are an expert Rust compiler repair agent..." }
      ]
    }
  ]
}
```

This is the exact format the Gemini API expects — the `contents` array with `parts` containing the text.

Making the HTTP Request with `request`:

```
let client = request::blocking::Client::new();
let res = client
    .post(&url)
    .header("Content-Type", "application/json")
    .json(&payload)
    .send()
    .map_err(|e| format!("HTTP request error: {}", e))?;
```

	Explanation		
	Create an HTTP client. `request` is an external crate for making HTTP requests. `blocking` means it's synchronous (waits for response). The async version would need `tokio` runtime.		
	This is an HTTP POST request. Gemini's API requires POST.		
	Tell the API we're sending JSON. Required by all REST JSON APIs.		
	Serialize `payload` to JSON and set it as the request body. `request` + `serde_json` handle the serialization.		
	Actually send the HTTP request. Returns `Result`.		
	e\	...)	Convert `req
	If network fails, propagate the error up.		

Parsing the API Response:

```
let json_resp: serde_json::Value = res.json()
    .map_err(|e| format!("Failed to parse Gemini response JSON: {}", e))?;

let text = json_resp["candidates"][0]["content"]["parts"][0]["text"]
    .as_str()
    .unwrap_or("")
    .to_string();
```

Code	Explanation
<code>res.json()</code>	Parse the HTTP response body as JSON into a generic <code>serde_json::Value</code> .
<code>json_resp["candidates"][0]["content"]["parts"][0]["text"]</code>	Navigate the JSON object using index operators. Gemini's response structure: <code>candidates[0].content.parts[0].text</code> contains the generated text.
<code>.as_str()</code>	Try to interpret the JSON value as a string. Returns <code>Option<str></code> .
<code>.unwrap_or("")</code>	If the field is missing, use empty string.

The Gemini API response looks like:

```
{
  "candidates": [{
    "content": {
      "parts": [{
        "text": "Here is the repaired code:\n```\nrust\npub fn swap(...) { ... }\n```"
      }]
    }
  ]
}
```

Lines 133-146: The Code Block Extractor

```
fn extract_rust_code_block(text: &str) -> Option<String> {
    if let Some(start) = text.find("```rust") {
        let after_start = &text[start + 7..]; // Skip "```rust" (7 chars)
        if let Some(end) = after_start.find("```") {
            return Some(after_start[..end].trim().to_string());
        }
    } else if let Some(start) = text.find("```") {
        let after_start = &text[start + 3..]; // Skip "```" (3 chars)
        if let Some(end) = after_start.find("```") {
            return Some(after_start[..end].trim().to_string());
        }
    }
    None
}
```

How it works on this input:

```
"Here is the repaired code:\n```rust\npub fn swap(...) {} \n```\nHope this helps!"
```

Step	Code	Value
Find opening	<code>text.find("```rust")</code>	<code>Some(28)</code> — offset of the backticks
Skip marker	<code>start + 7</code>	35 — <code>```rust`</code> is 7 characters
Slice after	<code>&text[35..]</code>	<code>"\npub fn swap(...) {} \n```\nHope..."</code>
Find closing	<code>.find("```")</code>	<code>Some(21)</code> — the closing triple-backtick
Extract	<code>after_start[..21]</code>	<code>"\npub fn swap(...) {} \n"</code>
Clean	<code>.trim()</code>	<code>"pub fn swap(...) {}"</code>

The `else if` handles responses without the `rust` language specifier (just ```` ` ````).

Lines 109-131: The Rule-Based Fallback

```
fn fallback_rule_based_repair(pkg: &RepairRequestPackage) -> PatchProposal {
    let mut patched = pkg.candidate_rust_code.clone();
    let lines: Vec<&str> = patched.lines().collect();
    let mut clean_lines = Vec::new();

    for line in lines {
        let trimmed = line.trim();
        if trimmed.starts_with("void ") || trimmed.starts_with("int main()") {
            continue; // Strip accidentally-included C function headers
        }
        clean_lines.push(line);
    }

    patched = clean_lines.join("\n");

    PatchProposal {
        patched_rust_code: patched,
        explanation: "Applied syntax normalization and rule-based repair fallback.".to_string(),
        source: "Rule-Based-Fallback".to_string(),
    }
}
```

The fallback is intentionally SIMPLE — it handles common artifacts where C function signatures accidentally leaked into the Rust output. It can't fix type errors or borrow checker violations (that requires the LLM), but it can handle the most common syntactic issues. Returns a `PatchProposal` (not `Result`) because it always succeeds.

File 5: `src/module6_repair/patch_applier.rs`

(Writing the Repair Back to Disk)

Lines 4-10: The `RepairLogEntry` Struct

```
#[derive(Debug, Clone)]
pub struct RepairLogEntry {
    pub iteration: usize,
    pub patch_source: String,
    pub explanation: String,
    pub compilation_successful: bool,
}
```

Field	What it holds
<code>`iteration`</code>	Which iteration produced this log (1, 2, 3...).
<code>`patch_source`</code>	<code>`"Gemini-LLM-API"`</code> or <code>`"Rule-Based-Fallback"`</code> .
<code>`explanation`</code>	What the LLM (or fallback) said it did.

Field	What it holds
<code>`compilation_successful`</code>	Whether the NEXT compilation after this patch succeeded. Set to <code>`false`</code> initially by <code>`apply_patch_to_file`</code> , then updated by the main loop if successful.

Lines 12-28: The `apply_patch_to_file()` Function

```
pub fn apply_patch_to_file(
    rs_file_path: &str,
    proposal: &PatchProposal,
    iteration: usize,
) -> Result<RepairLogEntry, String> {
    fs::write(rs_file_path, &proposal.patched_rust_code)
        .map_err(|e| format!(
            "Failed to apply patch to file '{}': {}",
            rs_file_path, e
        ))?;

    Ok(RepairLogEntry {
        iteration,
        patch_source: proposal.source.clone(),
        explanation: proposal.explanation.clone(),
        compilation_successful: false, // Will be updated by the main loop
    })
}
```

	Explanation		
<code>code)`</code>	OVERWRITE the file with the repaired code. <code>`fs::write`</code> replaces the entire file content.		
<code>e\</code>		<code>format!(...))`</code>	If writing fails
	Propagate write failure up to <code>`run_repair_loop`</code> , which will stop the loop.		
	Always set to <code>`false`</code> here — we haven't compiled yet. The main loop in <code>`mod.rs`</code> will set it to <code>`true`</code> if the NEXT <code>`invoke_rustc_compiler`</code> call succeeds.		

The Complete Module 6 Data Flow with Examples

```

Iteration 1:
invoke_rustc_compiler("output/output.rs")
-> CompilationResult {
    success: false,
    error_count: 3,
    diagnostics: [
        RustcDiagnosticMessage { level: "error", code: Some(Code{"E0382"}),
                                message: "use of moved value: `buf`", spans: [...] }
        ...
    ]
}

package_diagnostics(c_src, rust_src, diagnostics, 1)
-> RepairRequestPackage {
    compiler_error_messages: ["[E0382] use of moved value: `buf`", ...]
    original_c_source: "void alloc_buf(int **out) { *out = malloc(100); }"
    candidate_rust_code: "pub fn alloc_buf(out: &mut Box<i32>) { ... }"
    iteration_number: 1
}

query_llm_repair_agent(pkg)
-> (HTTP POST to Gemini API with prompt containing all three sections)
<- Gemini responds: "The issue is that buf is moved. Here is the fix:

```

```

pub fn alloc_buf(out: &mut Option<Box<i32>>){
*out = Some(Box::new(0));
}

```

```

extract_rust_code_block(response_text)
-> "pub fn alloc_buf(out: &mut Option<Box<i32>>) { ... }"

apply_patch_to_file("output/output.rs", proposal, 1)
-> Writes repaired code to disk
-> RepairLogEntry { iteration: 1, source: "Gemini-LLM-API", ... }

```

```

Iteration 2:
invoke_rustc_compiler("output/output.rs")
-> CompilationResult { success: true, error_count: 0 }
-> RETURN RepairResult { final_success: true, iterations_used: 2, ... }

```

Key Rust Concepts Introduced in This Module

	Example	Meaning		
	On diagnostic structs	Auto-generate JSON serialization code via the `serde` crate.		
:(line)`	Parse JSON to struct	Deserialize a JSON string into a specific Rust type.		

	Example	Meaning		
	Inclusive range	<code>`..=`</code> includes the last value. <code>`1..=3`</code> = <code>`[1, 2, 3]`</code> .		
	Early exit from function	Exits the entire function from inside a loop — not just the loop.		
	Read env variable	Returns <code>`Result`</code> . <code>`ok()`</code> converts to <code>`Option`</code> .		
	<code>`\`</code>	<code>expr)`</code>	Option lazy fallback	If <code>`Option`</code> is closure and
client`	HTTP client	Synchronous HTTP from the <code>`request`</code> crate.		
	Serialize body	<code>`request`</code> + <code>`serde`</code> auto-serialize the payload to JSON.		
	Generic JSON	A Rust type that can hold any JSON value — navigated with <code>`["key"]`</code> and <code>`[0]`</code> indexing.		
	JSON macro	Inline JSON construction in Rust code. From <code>`serde_json`</code> crate.		
	<code>`diagnostics: &[RustcDiagnosticMessage]`</code>	More general than <code>`&Vec;`</code> — works with any contiguous sequence.		
	<code>`d.code.as_ref()`</code>	Borrow the inside of <code>`Option`</code> to get <code>`Option<&T;>`</code> without moving.		
	<code>`Option::unwrap_or_default()`</code>	Returns <code>""</code> for <code>`Option`</code> , <code>`0`</code> for <code>`Option`</code> , etc.		
	<code>`Option`</code>	Find a substring's start position.		
	Slice from offset	Take a string slice starting at byte offset <code>`start + 7`</code> .		
l: false`	Struct init	Set a field to known default; caller will update it.		

Summary

Module 6 is the **AI-powered quality gate** — it takes the generated Rust code and iteratively repairs it until it compiles:

`compiler_loop.rs` — Runs `rustc --error-format=json --crate-type=lib` and uses `serde` to parse each JSON diagnostic line into structured `RustcDiagnosticMessage` structs. Returns `CompilationResult` with success flag, error count, and all diagnostics.

`diagnostic_packager.rs` — Bundles the original C source, broken Rust code, and human-readable error strings into a `RepairRequestPackage` — the complete context payload for the LLM.

`llm_client.rs` — The AI integration heart. Reads the API key from environment variable or `.env` file, constructs a structured prompt with three context sections (C source, Rust code, errors), makes an HTTP

POST request to the Gemini API using `reqwest`, parses the JSON response, and extracts the repaired code from the ``rust`` block. Falls back to rule-based repair if the API is unavailable.

`patch_applier.rs` — Writes the repaired code over the candidate `.rs` file using `fs::write` and creates a `RepairLogEntry` for the GUI audit trail.

`mod.rs` — Orchestrates the loop: compile → package → query → patch → repeat. Returns `RepairResult` with final success status, iteration count, all log entries, and the final Rust code.

Document generated for C2SafeRust-LLM Mini Project S3

Module 6: Compiler Validation & LLM Repair Loop — Complete Line-by-Line Explanation